



C# 프로그래밍 첫걸음

완벽한 실습 환경 구축 가이드:
Visual Studio 2022 설치부터 첫 코드 실행까지

초보자를 위한 튜토리얼

오늘의 학습 여정



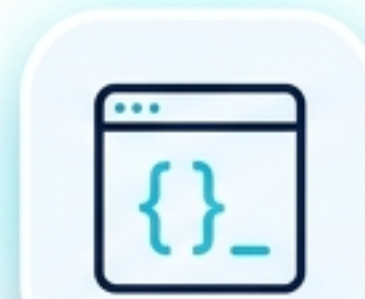
1. 환경 구축

Visual Studio 2022 설치



2. 프로젝트 생성

내 생애 첫 C# 앱



3. 코드 작성 및 실행

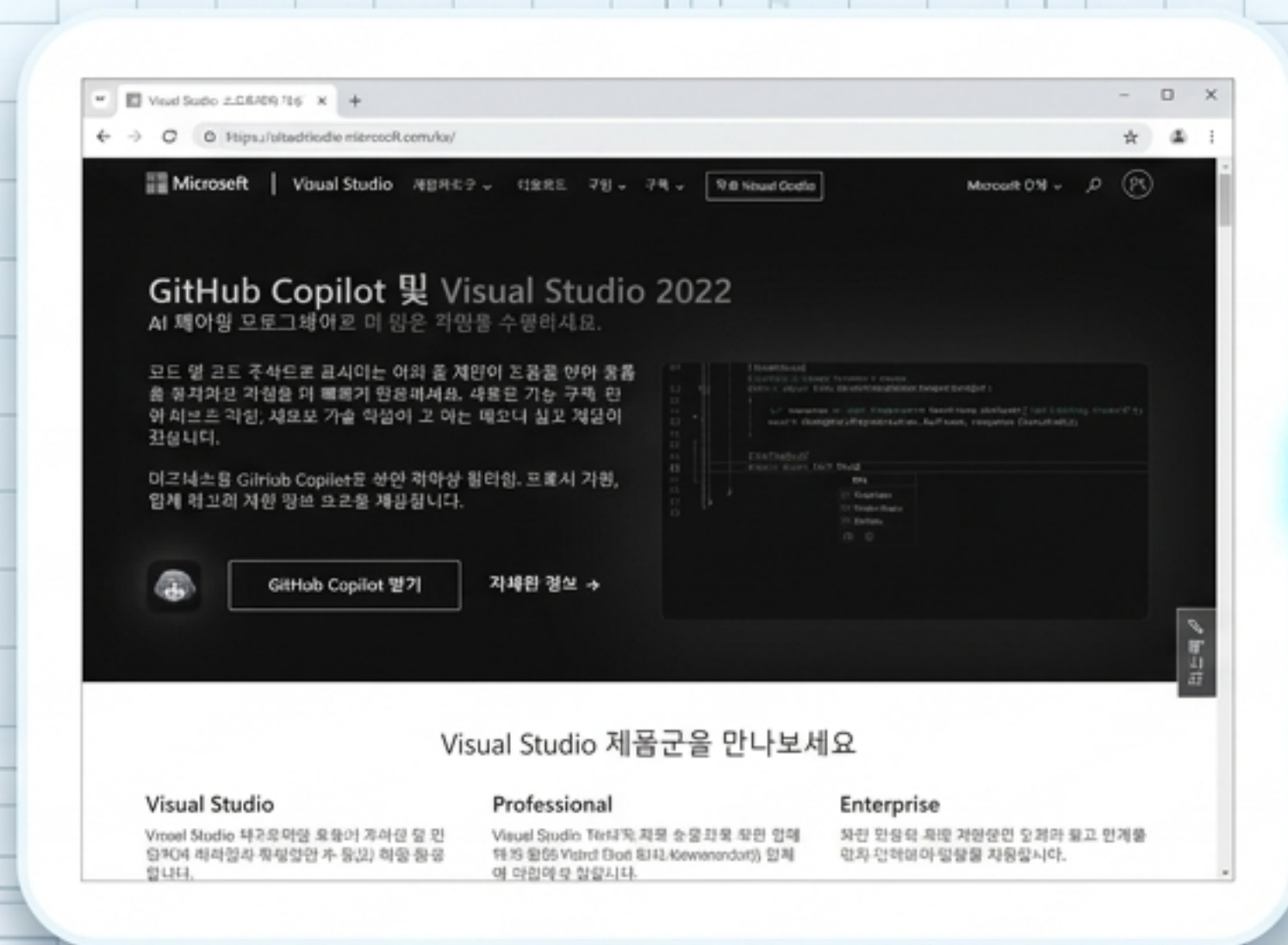
Write와 WriteLine의 이해



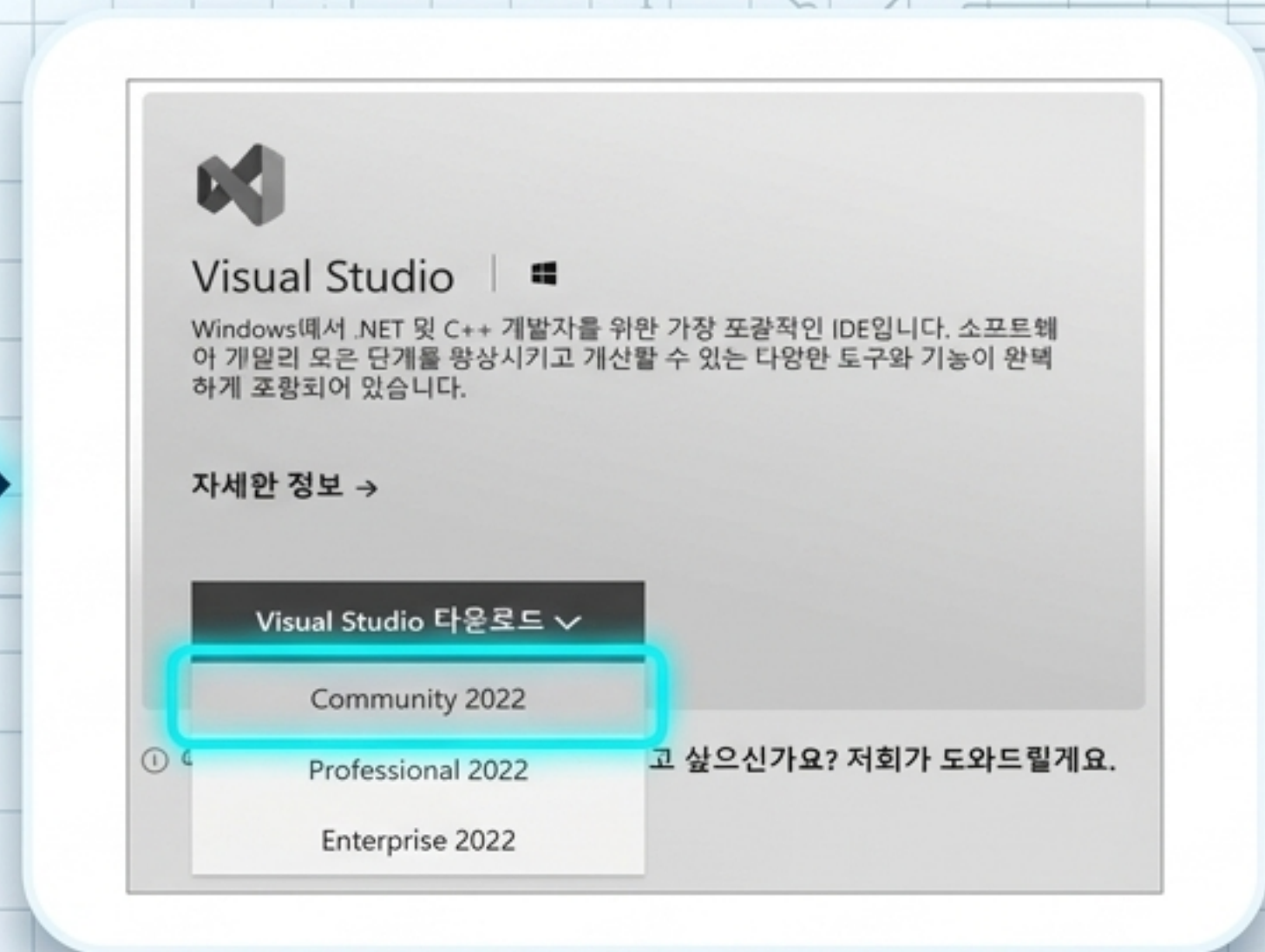
4. C# 핵심 개념

코드 구조와 언어의 진화

Step 1. Visual Studio 2022 다운로드



공식 웹사이트(visualstudio.microsoft.com) 접속

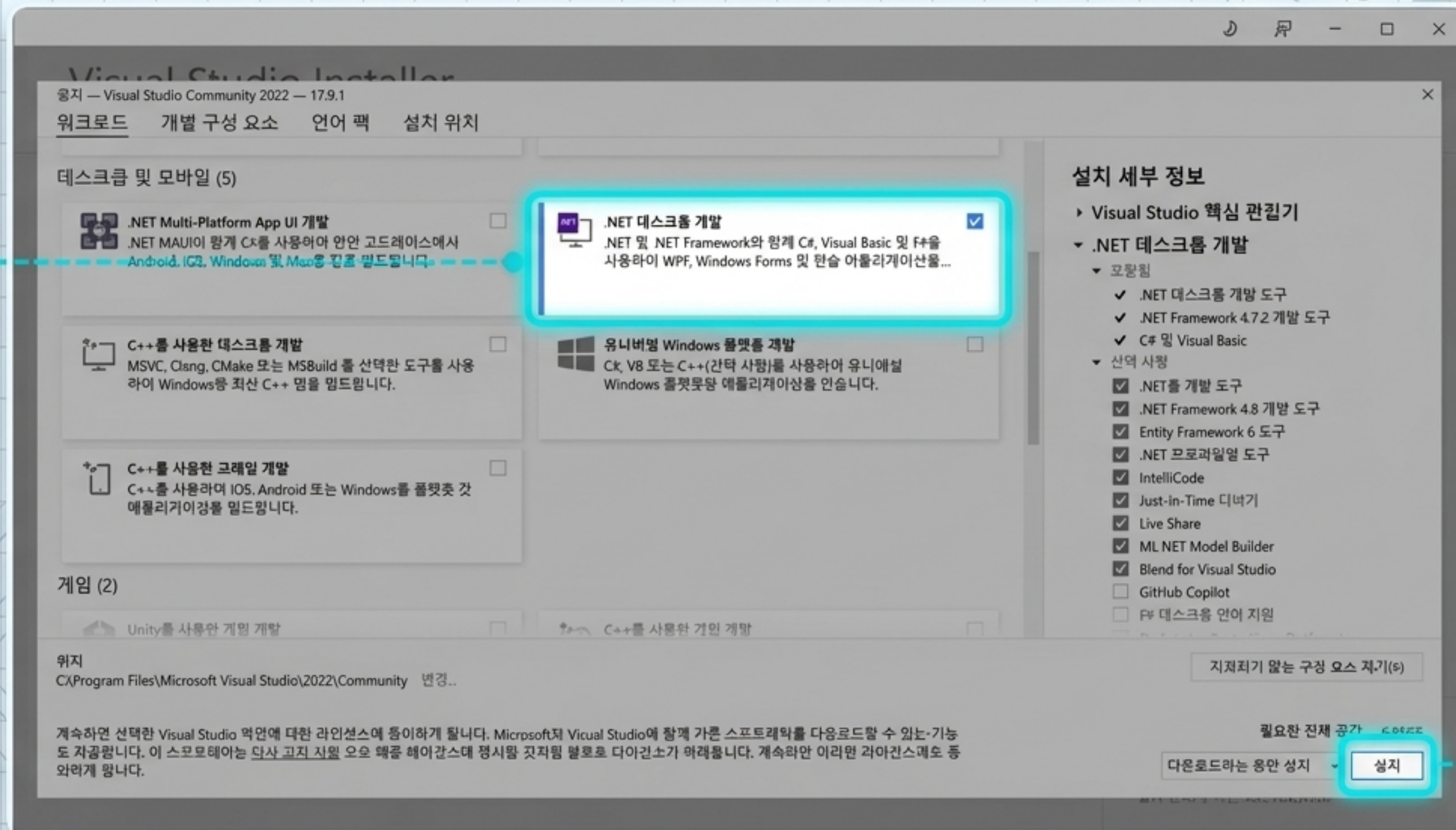


개인 학습자를 위한 무료 통합 개발 환경(IDE)인
Community 2022 버전을 선택하세요.

Step 2. 필수 워크로드 선택 (가장 중요!)

⚠ 이 단계를 놓치면
C# 코드를 작성할 수 없습니다.

.NET 데스크톱
개발 항목을 반드시
체크하세요.
C# 프로그래밍에
필요한 모든
도구가 포함되어
있습니다.



체크 후
우측 하단의
설치 버튼 클릭.

Step 3. 첫 실행 및 개발자 테마 설정



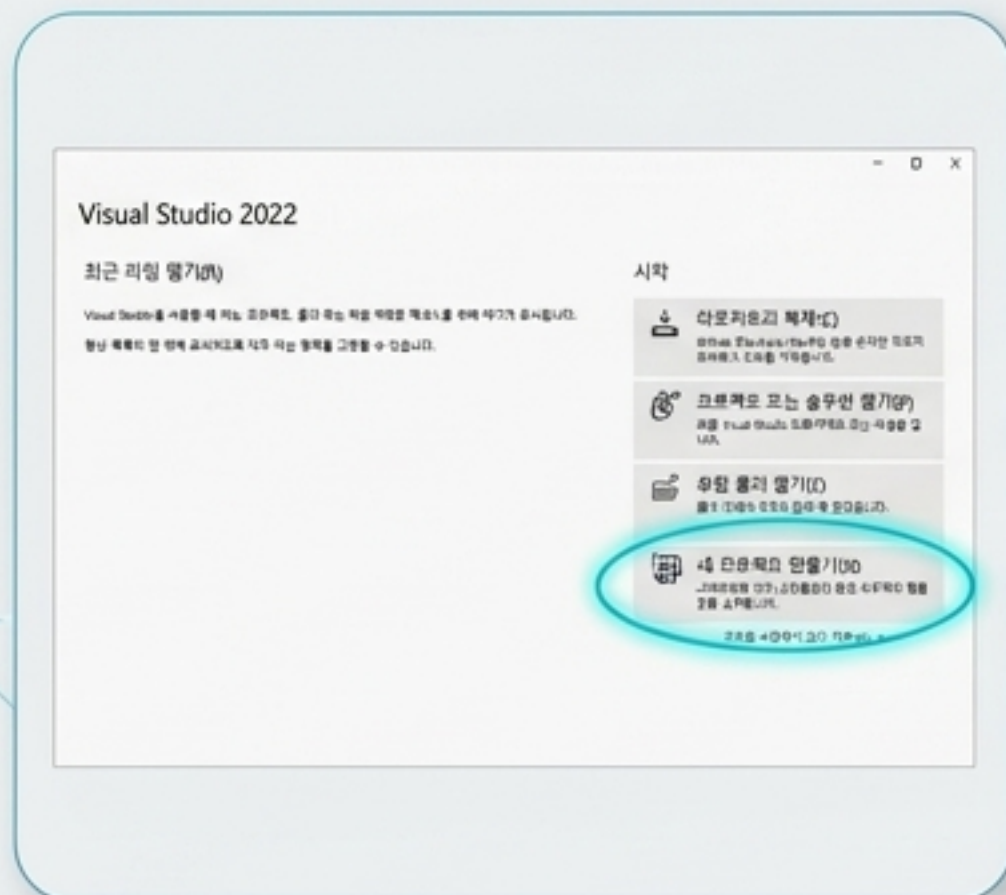
설치 완료 후 나타나는 Visual Studio 2022 스플래시 스크린.



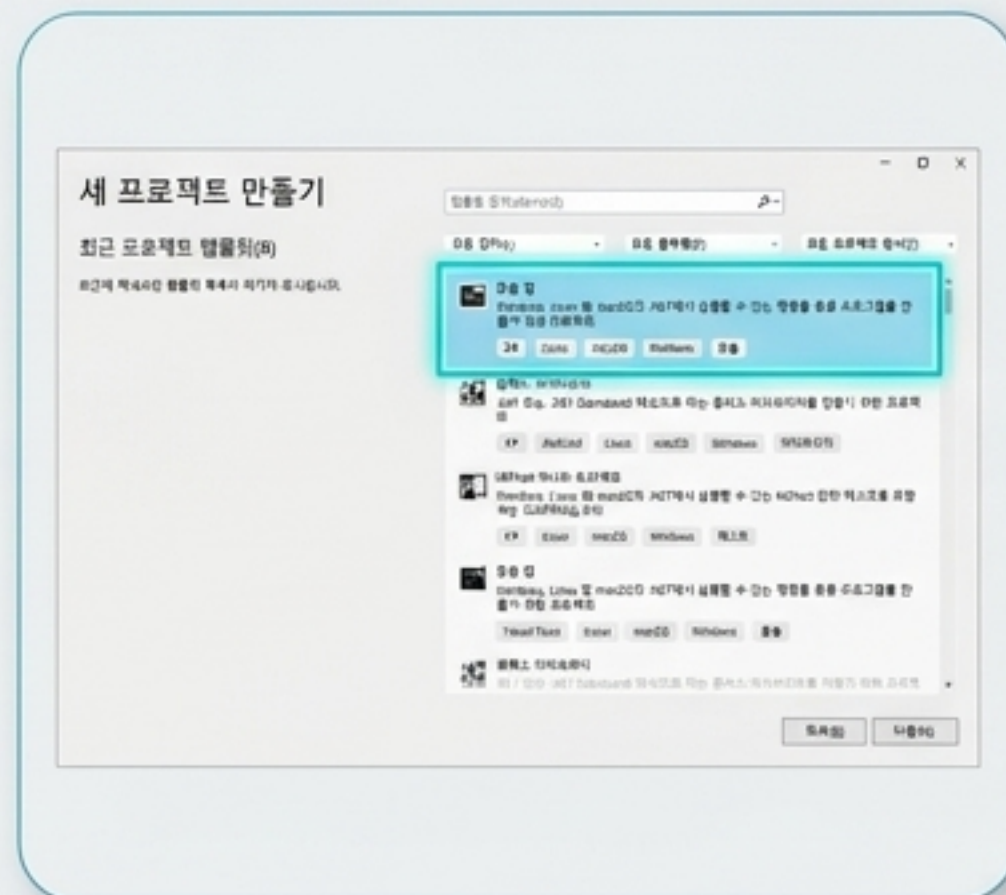
개발자들이 가장 선호하는 '어둡게 (Dark Mode)' 테마를 추천합니다. 눈의 피로를 덜어줍니다.

[Visual Studio 시작] 클릭!

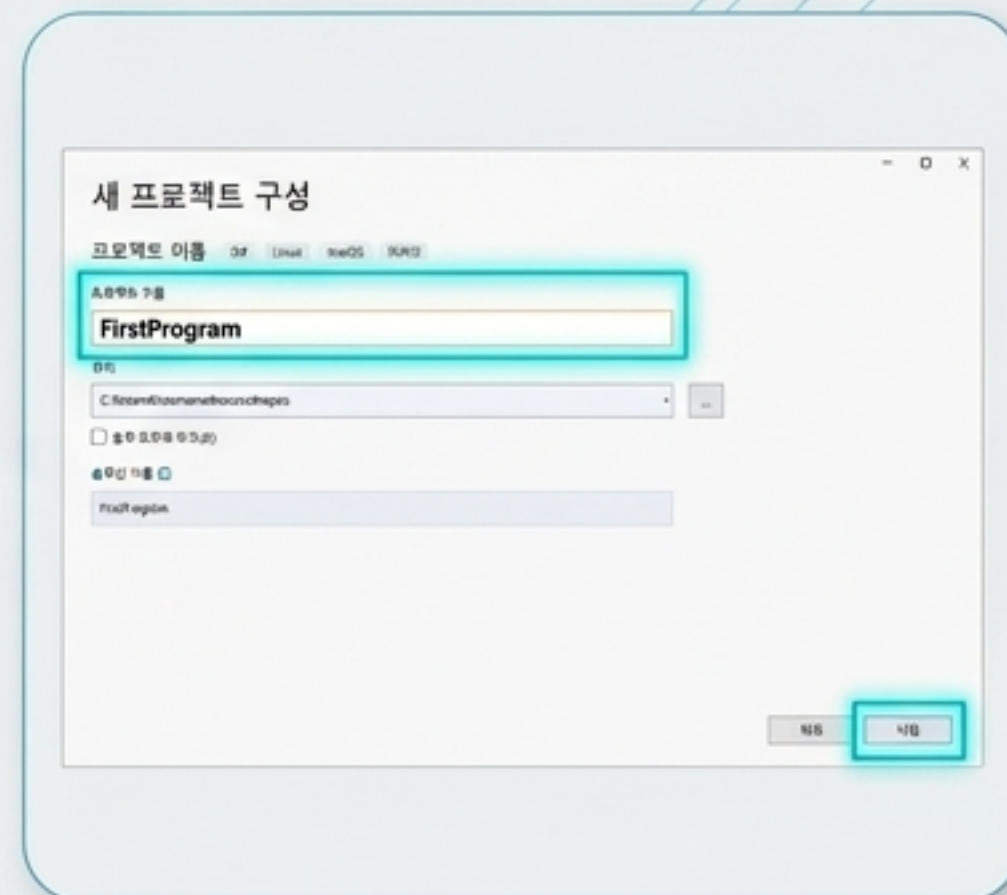
Step 4. 첫 번째 프로젝트 생성하기



새 프로젝트 만들기 클릭



C# 콘솔 앱 템플릿 선택
(명령줄 기반의 기본 프로그램)



프로젝트 이름에 **FirstProgram**
입력 후 다음 클릭

잠깐! 기본기를 위한 필수 설정

Framework

NET 6.0 (장기 지원)

☐ 솔루션 및 프로젝트를 같은 디렉터리에 배치

☒ 최상위 문 사용 안 함

[최상위 문 사용 안 함]
옵션을 반드시 체크하세요.

최신 C#은 코드를 극단적으로 줄여주는
'최상위 문' 기능을 제공합니다.
하지만 초보자 단계에서는 C#의
전통적인 기본 구조를 먼저 눈으로 보고
이해하는 것이 매우 중요합니다.

개념 체크: 솔루션(Solution) vs 프로젝트(Project)

솔루션 (예: 리그 오브 한빛 게임 전체) - 관련된 여러 프로젝트를 하나로 묶어서 관리하는 거대한 컨테이너.

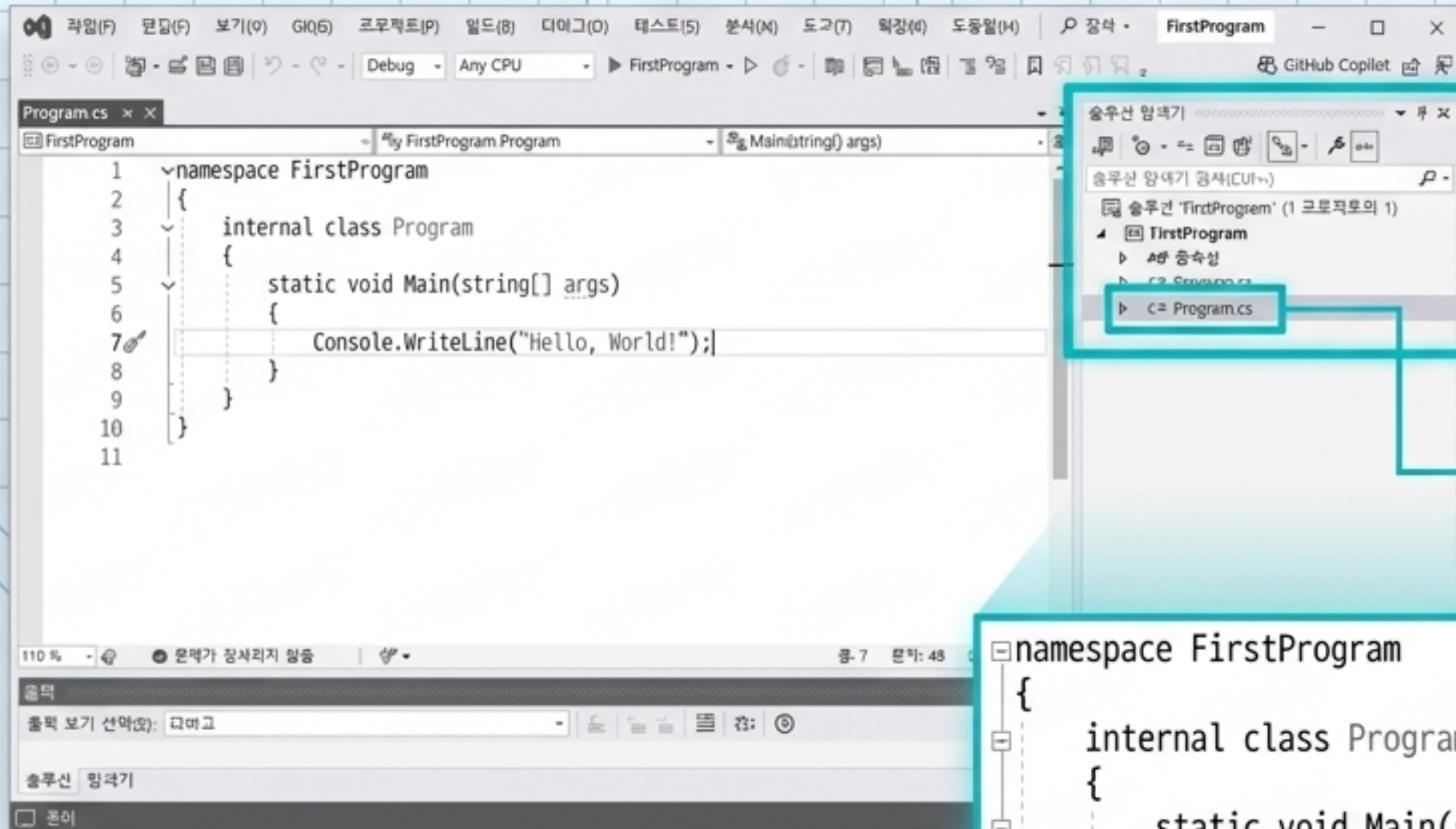
프로젝트 A (서버 시스템)

프로젝트 B (클라이언트 UI)

프로젝트 C (맵 관리 프로그램)

방금 우리가 만든 것은 하나의 '프로젝트'이며, Visual Studio는 이를 담기 위한 '솔루션'을 자동으로 함께 생성합니다.

작업 공간 해부도 (Anatomy of the Workspace)



솔루션 탐색기 - 프로젝트에 포함된 파일들을 보여줍니다.

Program.cs - 우리 프로그램의 심장이자, 코드를 작성하는 메인 파일입니다.

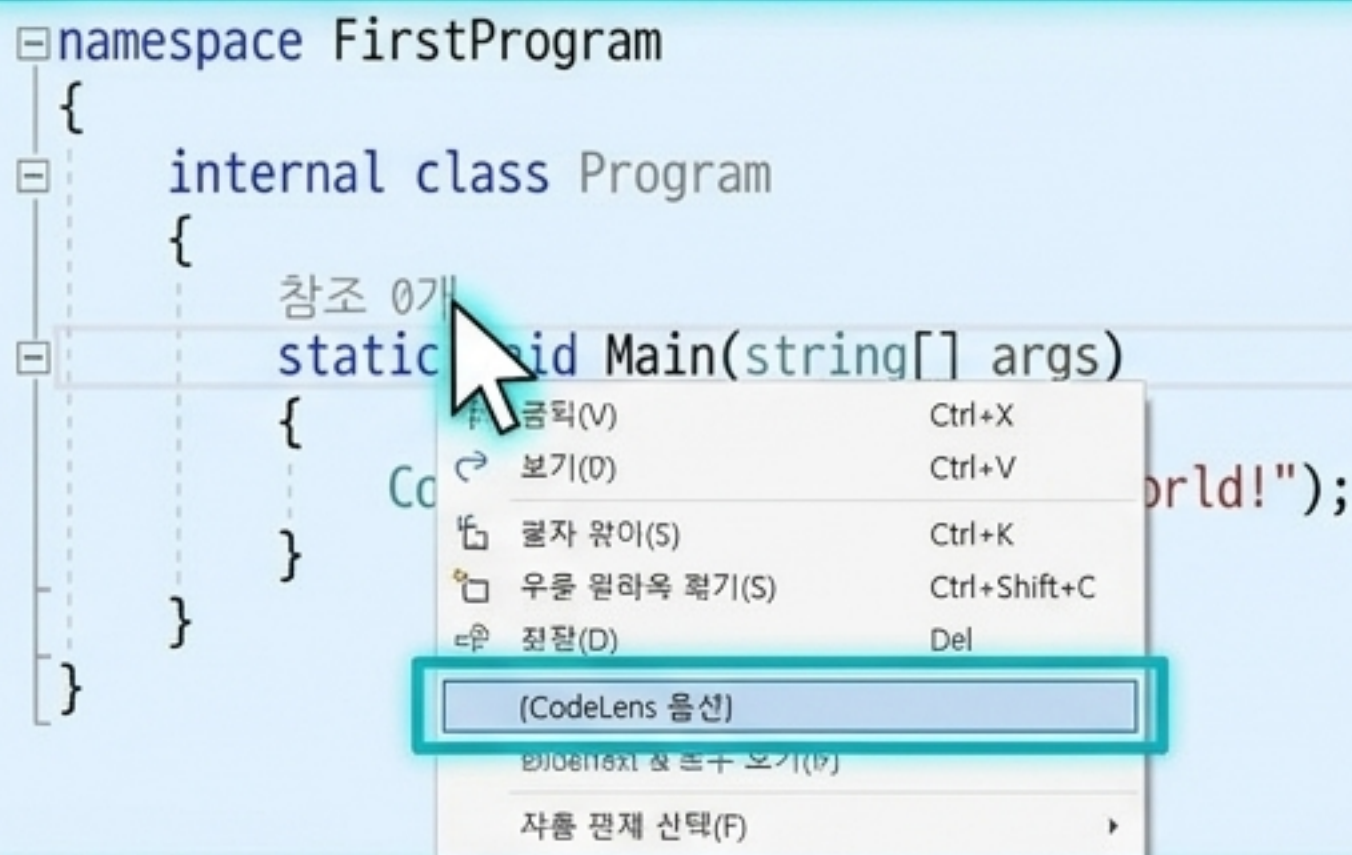
```
namespace FirstProgram
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

기본 구조(namespace, class, Main)가 이미 작성되어 있습니다. 컴퓨터는 항상 Main 안쪽의 코드부터 실행합니다.

초보자를 위한 화면 최적화 (CodeLens 끄기)

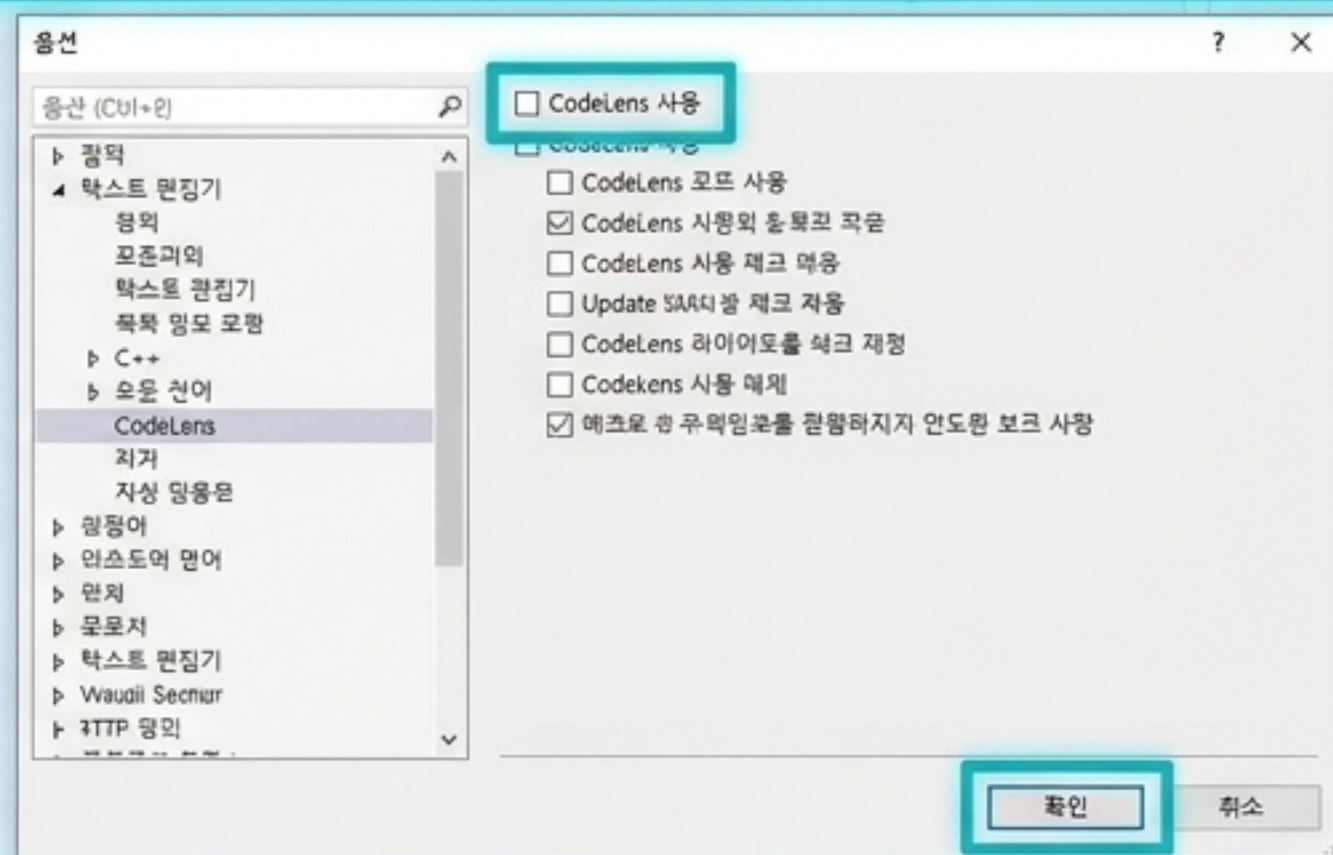
코드 위에 '참조 0개'라는 불필요한 글자가 보인다면?

Step 1



1. 글자 위 우클릭 → [CodeLens 옵션] 선택.

Step 2



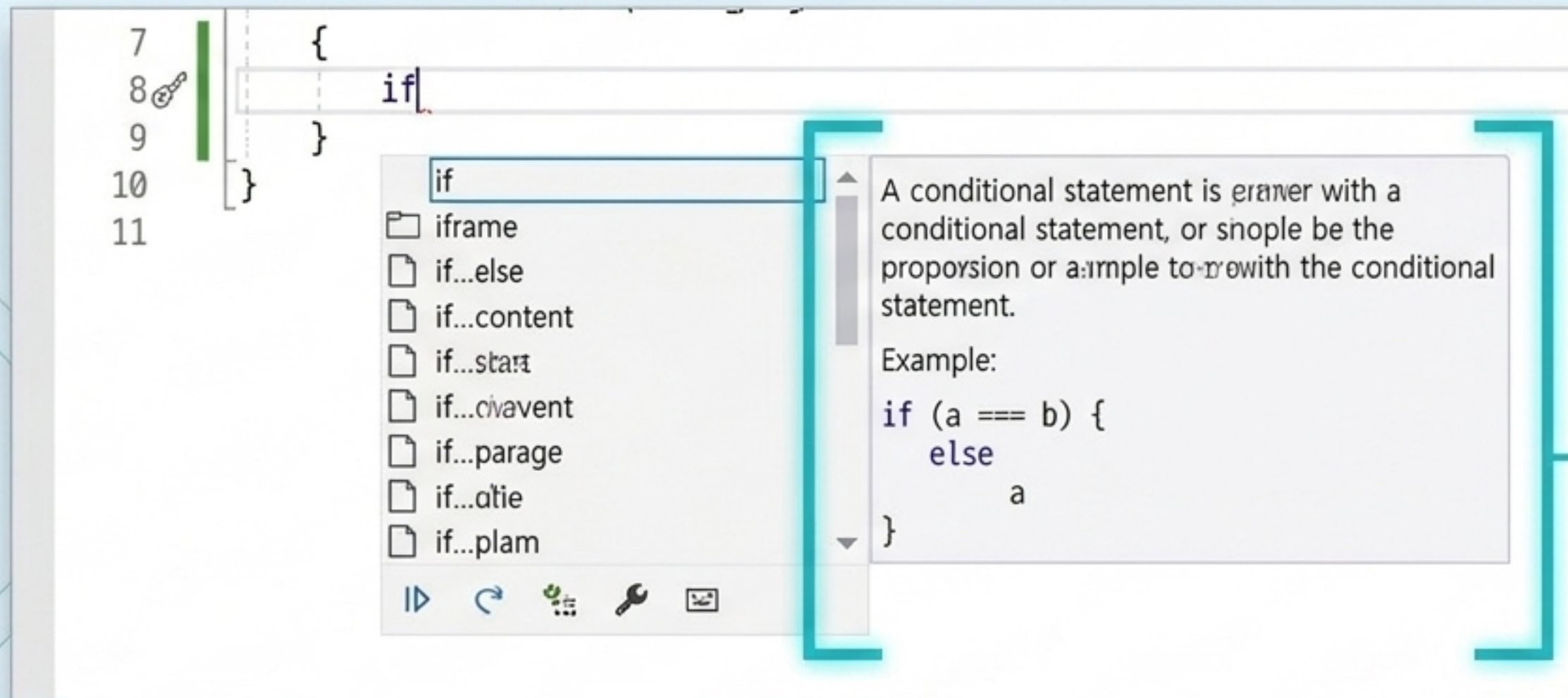
2. 옵션 창에서 [CodeLens 사용] 체크 해제 후 확인.

화면이 훨씬 깔끔해져 코드 자체에만 집중할 수 있습니다.

프로그래밍을 잘하는 습관: 자동 완성 (IntelliSense)

마법의 단축키
[Ctrl] + [Space]

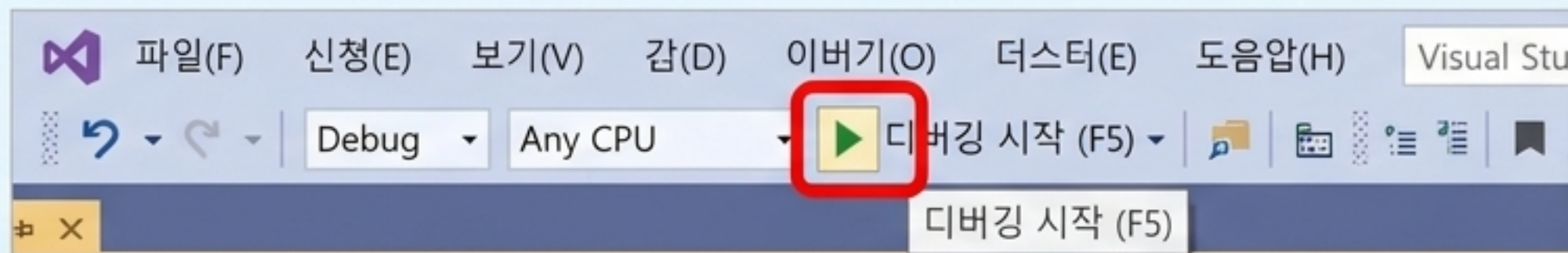
타이핑 중 언제라도 단축키를 누르면 Visual Studio가
사용 가능한 명령어 목록을 띄워줍니다.



단순한 자동 완성이 아닙니다.
화살표를 위아래로 움직이면
해당 명령어가 어떤 역할을
하는지 사전처럼 친절한 설명
을 볼 수 있습니다.

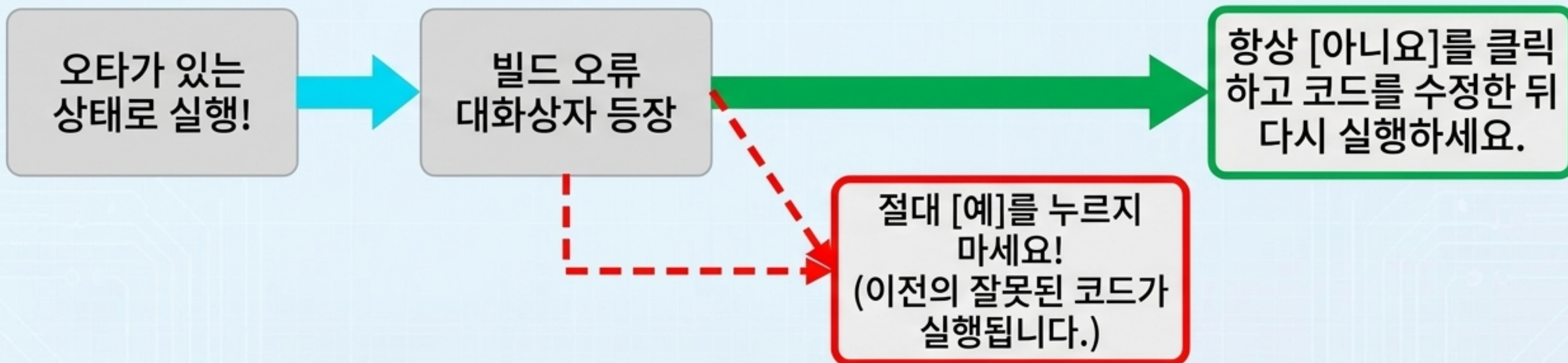
코드 실행과 치명적인 함정 피하기

How to run



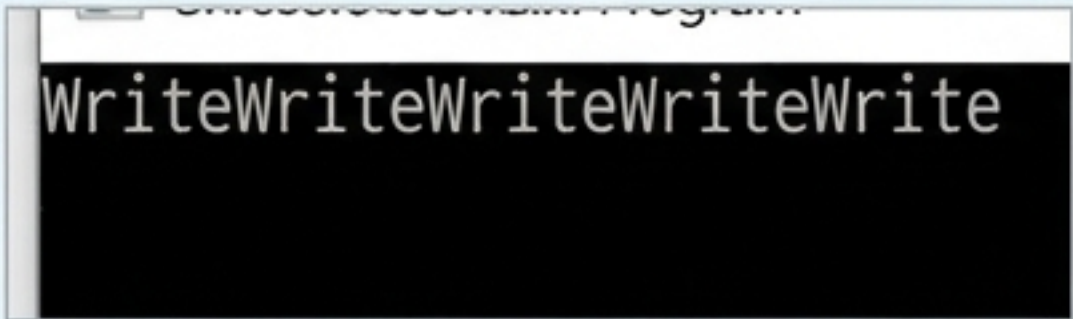
상단의 [디버깅 시작] 버튼 클릭
또는 단축키 [F5] 누르기.

Error Flowchart



출력의 기술: Write() vs WriteLine()

```
Console.Write("Write");  
Console.Write("Write");  
Console.Write("Write");  
Console.Write("Write");  
Console.Write("Write");
```



WriteWriteWriteWriteWrite

Console.Write() - 문자를 출력한 뒤
그 자리에 머물습니다.

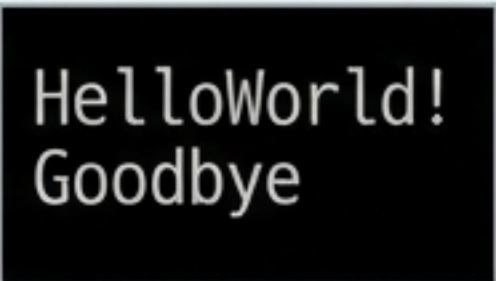
```
Console.WriteLine("Write");  
Console.WriteLine("Write");  
Console.WriteLine("Write");
```



Write
Write
Write

Console.WriteLine() - 문자를 출력한
뒤 엔터(줄바꿈)를 칩니다.

```
Console.Write("Hello");  
Console.WriteLine("World!");  
Console.Write("Goodbye");
```



HelloWorld!
Goodbye

Line이 붙으면 ‘출력 후 줄을
바꾼다’는 의미입니다.

언어의 진화: 왜 이렇게 코드가 길었을까?

프로그래밍 초창기에는 단 한 줄의 글자를 화면에 띄우기 위해서도 복잡한 '준비 작업(구조화)'이 필수적이었습니다.

C Language

```
#include <stdio.h>

int main() {
    printf("Hello, world!\n");
    return 0;
}
```

C 언어: #include, int main() 등
출력과 무관한 배경 설정이 많습니다.

```
using System;

namespace HelloWorld
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, world!");
        }
    }
}
```

전통적인 C#: C# 역시 namespace와
class라는 강력하지만 무거운 껍질로
핵심 출력 코드를 감싸야만 했습니다.

모던 C#의 시대: 최상위 문 (Top-Level Statements)

파이썬 등 스크립트 언어

```
print("Hello, World!")
```

가볍고 직관적입니다.

모던 C# 10.0+

```
Console.WriteLine("Hello, World!");
```

C# 9.0 이후, 파이썬처럼 복잡한 껍질을 생략하고
핵심 로직만 바로 작성할 수 있는 ‘최상위 문’ 기능이 도입되었습니다.

이제 여러분은 C#의 과거(기본 구조)와 현재(최상위 문)를 모두 이해했습니다.
완벽한 환경이 준비되었으니, 본격적인 코딩을 시작해 봅시다!